The Machine-Learning Layers of the Pipeline

What each layer is, what it does, and how it was tested

Explained edition, companion to "From Noisy Data to a Distribution over Equations"

October 2026

Abstract

The pipeline that turns noisy measurements of a linear equation into a distribution over equations contains machine learning in two different ways. *First*, classical statistical learning sits inside the main pipeline: a Gaussian process to interpolate, a Dirichlet-process mixture and clustering to group, a Bayesian linear model to split components, and an evidence-based decider with a refinement fit. These are implemented, committed and benchmarked. *Second*, a set of optional *learned layers* (Phase 4) was built in a separate session: learned seeds for the exponential fit, calibration of probabilities and intervals, a failure validator with an out-of-distribution gate, and a remedy table. These were built and then evaluated on corpora of simulated equations by a separate session: none improved the pipeline (the failure detector was refused for lack of detection power, the calibrations had nothing to calibrate), and the only layer that is on, an exact cache, changes no output. In the final benchmark the pipeline with and without the learned layers is the same 30/30. This document says, layer by layer, what it is, what it takes in and puts out, how it was tested (which tests, which experiments, which numbers), and what is not established.

Contents

1 The map: every learned or statistical-learning layer **2**

2 The rule every learned layer obeys, and how tests are built **2**

3 Machine learning inside the main pipeline **5**

3.1 The Gaussian process: interpolation with probabilities 5

3.2 The Dirichlet-process mixture and clustering 6

3.3 The Bayesian split layer 6

3.4 The decider, the refinement and the loop 7

4 The optional learned layers (Phase 4) **7**

4.1 The simulator: the ground where learned layers are trained and judged 8

4.2 L0: learned seeds for the exponential fit 8

4.3 L2: calibration of probabilities and intervals 9

4.4 V0 and V1: the validator 10

4.5 The remedy table 11

4.6 What was not built, and why 11

5 The final evaluation of the learned layers **11**

6 Test inventory **13**

7 What is established and what is not **13**

About this explained edition

It follows the format of the main explained edition: each section opens with a box *In this section* and a box *Symbols used here: what they are and where they come from*; **purple "Explained" boxes** read the material step by step; orange boxes give intuition; red boxes are warnings. Status labels: [implemented, tested] (code and tests exist, committed), [implemented, uncommitted] (code and tests exist, not committed), [prototype] (an experiment script only), [planned] (a plan only), [open] (open question). Grey tags such as (theory T10) point to rows of the theory-to-algorithm table; Appendix **??** of the main edition states them. Numbers come from the repository's logs and reports; where a number could not be checked it is said so.

1 The map: every learned or statistical-learning layer

In this section

Starting from	the pipeline of the main edition: scattered data, interpolation, spectral core, aggregation, grouping, Bayesian output, loop.
Why this section	the reader asked for all the machine-learning layers that were built, not only the optional ones. A map first avoids mixing what is in the main pipeline with what is optional.
Role in the paper	this section is the table of contents of the project from the point of view of learning; each row is explained in its own section.

Symbols used here: what they are and where they come from

GP	Gaussian process: a probability distribution over functions	Sec. 3.1
DP mixture	Dirichlet-process mixture: a clustering model with an unbounded number of clusters	Sec. 3.2
evidence	the probability of the data under a model, with the parameters integrated out	Sec. 3.3
proposal	a candidate offered to the exact evidence, which scores it	Sec. 2
calibration	an offline correction of a probability or an interval	Sec. 4.3
flag	a warning attached to a result; it never changes the label	Sec. 4.4
LOFO	leave-one-family-out: hold out a whole family of equations when testing	Sec. 2

Layer	Kind of learning	Code	Status
Interpolator	Gaussian process, hyperparameters marginalised, pathwise samples	`interp/gp.py`	[implemented, tested]
Factor-wise GP	GP with the factors' exponentials as features (EPGP)	`interp/epgp.py`	[implemented, tested] (used by the evidence)
Fast GP, ensemble	KISS-GP on a grid; deep ensemble	`interp/ski.py`, `ensemble.py`	[implemented, tested] (ablation, experimental)
Aggregation	Dirichlet-process mixture, merge step, finite-mixture BIC	`aggregate.py`	[implemented, tested]
Grouping	GMM, k-means, spectral, agglomerative on window features	`cluster.py`	[implemented, tested]
Bayesian split	linear-Gaussian model, empirical Bayes, closed-form evidence	`bayes_split.py`	[implemented, tested]
Decider, refinement	model selection by raw-data evidence, variable projection	`loop.py`, `refine_raw.py`	[implemented, tested]
Baselines	sparse regression (SINDy variants), DMD, for comparison	`baselines/`	[implemented, tested]
Simulator	random equations by family (training and test ground)	`simulator.py`	[implemented, tested] (held-out benchmark)
L0 seeds	CNN that proposes poles of an exponential sum	`learned/expsum.py`, `expseeds.py`	mechanism tested; no benefit result
L2 calibration	isotonic regression, monotone boosting, conformal intervals	`learned/calibrate.py`	no-go (Sec. 5)
V0 / V1 validator	deterministic diagnostics; boosted trees with risk control and an OOD gate	`validate.py`, `learned/validator.py`	V0 [implemented, tested]; V1 no-ship
Remedy table	offline contextual-bandit estimate	`learned/remedy.py`	unused (no controller)
L1b, L6, controller	set encoder; amortised posterior; remedy controller	none	not built
L5 encoder	learned grouping encoder	none	rejected

Table 1: All the layers. The first eight are the main pipeline; the next five are the optional learned layers.

Why it matters. The pipeline already contains real machine learning (a GP, a DP mixture, clustering, Bayesian regression). What was added in Phase 4 is an *optional* layer of neural and tree models, held to a stricter rule than the rest.

2 The rule every learned layer obeys, and how tests are built

In this section

Starting from	the principle that the analytic core is deterministic and the outer layer is Bayesian (the project rules).
Why this section	a neural network that can change a label or a likelihood would break the output contract; the rule says exactly what it may do.
Role in the paper	this is the template for every test in Section 4 and the reason several tests are of one kind, the "null gate".

Symbols used here: what they are and where they come from

off switch	a setting under which the pipeline is exactly the deterministic one	each layer
null gate	a test on data where the right answer is "nothing there": single pole, pure noise, single factor	Phase 4 plan
simulator prior	the distribution from which the training equations are drawn; outputs depend on it and carry a tag	`PRIOR_TAG`
family	one of five groups of random equations F1–F5	`simulator.py`
split (fit / calibrate / test)	three disjoint parts of the simulated corpus: one to fit, one to calibrate, one to report	Phase 4 plan

Theorem 2.1 (Rule of Phase 4) [implemented, tested]

Every learned output is a **proposal**, a **calibration** or a **flag**. The likelihood of the final comparison is fixed and global. Off equals today's pipeline bit for bit. The core changes only through an optional argument, with tests. A layer is switched on only if, on held-out families, it is at least as good as off in right set, coefficient error, coverage and false-confident rate, and passes the null gates.

Intuition

A learned component may change *where* the algorithm looks and *how fast* it gets there. It may never change *what the data say* about a candidate. This is why a seed from a network is scored by the same criterion as one from the matrix pencil.

Explained: how the tests are built, step by step

Step 1: the off test. Run the layer off and compare with a recorded output of the unmodified code. For the seed hook the comparison is against modes stored in `tests/data/structfit_golden.npz`, generated with the same data recipe from the previous version of the core, and they must be reproduced exactly.

Step 2: a guarantee that holds whatever the network says. For L0 the test feeds *junk* seeds and checks that the selected per-bin criterion can only go down, because a seed joins the comparison as one more candidate.

Step 3: the null gate. A single clean pole with junk seeds of every order must give no extra mode; pure noise must give no mode at all.

Step 4: reliability, coverage and monotonicity. For a calibration, simulate data with a known answer and check that the probability matches the frequency (reliability) and that intervals cover at the nominal rate.

Step 5: held-out families. The simulator draws equations by family; model selection never sees the family it is judged on (leave-one-family-out). A test checks that the groups used for splitting are disjoint.

Why it matters. The tests are not accuracy tests on one case. They check that the layer cannot do harm (off, junk, null) before checking that it helps.

3 Machine learning inside the main pipeline

In this section	
Starting from	the data (t_i, x_i, y_i) and the analytic core.
Why this section	the core needs a field on a grid with an error model; the data are scattered and noisy. The outer layers supply this and the grouping.
Role in the paper	these layers are committed and benchmarked; the optional layers of Section 4 are built on top of them.

3.1 The Gaussian process: interpolation with probabilities

Symbols used here: what they are and where they come from

kernel k	prior covariance between the values of f at two points	choice of the algorithm
R	number of separable terms in the kernel sum	choice
η	hyperparameters (scales, correlation lengths, noise)	marginalised
posterior sample	one field drawn from the posterior	pathwise conditioning
cov_{90}	fraction of the truth inside the nominal 90% band	test

What it does. A GP with a sum of separable kernels $k_t \otimes k_x$ (optionally with a time envelope that mimics $e^{\lambda t}$) gives many plausible fields $\hat{u}_s$ with weights π_s, instead of one smooth curve. Hyperparameters, noise included, are marginalised: type-II maximum a posteriori, a Laplace approximation and importance reweighting by default, NUTS for small samples, a random subset for large ones. Samples are drawn by pathwise conditioning (Matheron's rule). Alongside it: a factor-wise EPGP (features $e^{\lambda(\kappa)t+i\kappa x}$ on the factors' varieties, closed-form evidence), KISS-GP on a grid (experimental, not calibrated), a deep ensemble (ablation; known to be under-dispersed away from the data), and a bridge that runs the core on the samples.

Explained: how it was tested

Unit tests (`tests/test_interp.py`, 11 tests): shapes and weights, loose coverage, samples matching the marginals, the subset path for large n, the mixture probability-integral transform against SciPy, the smoother, the core bridge on transport plus diffusion, EPGP prior draws solving transport, EPGP features on the variety, EPGP component weights, an ensemble smoke test.

Coverage experiment. Nine named cases, $n \in \{400, 2000\}$, noise 1% and 5%, 54 fits for the default family set: the 90% band covers 0.93, 0.92, 0.90, 0.89 in the four cells; the median estimated noise is within 4% of the true one. The criterion (± 5 points at 50, 90, 95%) is met in 11 of 12 cells; the miss is conservative. An RBF family set fails at $n = 2000$ (cov_{90} 0.84–0.86): evidence prefers the smoothest kernel that fits, and that is overconfident, so the family *set* is a prior choice made by calibration.

First end-to-end test. The core on the samples reached only 0.40 weight on the right factors: the core then took a single white noise. This led to per-bin correlated noise in the core (`BinNoise`).

3.2 The Dirichlet-process mixture and clustering

Symbols used here: what they are and where they come from

$\mathcal{W}$	weighted cloud of factor coefficient vectors	Sec. 5 of the main edition
k_{eff}	effective number of factors, exp of the entropy of the masses	as above
γ_0	DP concentration, chosen by the variational bound	empirical Bayes
ARI	adjusted Rand index between two partitions	clustering evaluation

What it does. `aggregate.py` pools the factors of all samples into $\mathcal{W}$ and fits a truncated stick-breaking DP mixture per τ-degree (scikit-learn's Bayesian Gaussian mixture), with a merge step by BIC for clusters that never co-occur in a sample. `cluster.py` extracts analytic window features and clusters rectangles by four methods (GMM, k-means, spectral with adjacency, agglomerative with connectivity); the number of groups is the MAP of a BIC posterior. A dominant-balance baseline is included for comparison.

Explained: how it was tested

Unit tests (`tests/test_aggregate_cluster.py`, 11 cases): the factor vector is invariant to padding, the alignment is invariant to the order of factors, the aggregates of transport plus diffusion and of heat, rectangles and adjacency, the two-regime ARI for each method, the global $\mathcal{W}$ on the two-regime field, the dominant-balance baseline runs.

Results. $P(k = 2) = 1.0$ on transport plus diffusion with masses $(0.768, 0.232)$; $P(k = 1) = 1.0$ on heat after the merge step (margin 7.96 against 8.32, thin). ARI $= 1.0$ for all four methods when the windows are aligned with $x = \pi$; 0.63 for three of them when one window straddles the boundary; dominant balance 0.20–0.35. These runs used the exact components as input, so they show the method on clean separation only.

3.3 The Bayesian split layer

Symbols used here: what they are and where they come from

S, a	matrix of kernel bases and the vector of coefficients: $u = Sa + \varepsilon$	Sec. 7 of the main edition
ω^2	prior variance of the coefficients, by empirical Bayes	as above
c_F, γ_2	cosine of the Friedrichs angle; $\gamma_2 = 1 - c_F$	as above
$P(\text{split})$	posterior probability that the two-factor model beats the one-factor model	closed-form evidence

What it does. A linear-Gaussian model per Fourier bin gives credible intervals for the component weights (by Monte Carlo) and the probability that the split exists. It uses a real (conjugate-pair) parametrisation at $\kappa = 0$ and a per-bin correlated noise covariance from the interpolation draws. The same machinery supplies the *raw evidence*: the Gaussian evidence of the raw scattered observations in the span of the factors' kernels, used by the loop to choose the structure.

Explained: how it was tested

Unit tests (`tests/test_phase3.py`, 10 tests): the per-bin noise recovers an AR(1) autocovariance; generalised least squares with white draws equals the white core; the weights cover the truth and the null stays low; the decider rejects constant factors per κ, rejects a symbol that interpolates few modes, counts only observed branches, rejects an interpolation artefact; the distinctness uses interval overlap; conjugate pairs and duplicates merge; bootstrap intervals cover the heat coefficient.

Experiment E8. Noise 10^{-2}, true factors. Transport plus diffusion: coverage of the 90% interval 0.95–1.00 for $T \in [0.25, 2]$; wave: 0.85–1.00; null (heat with a spurious transport): $P(\text{split}) = 0.000$ mean and max. The first version covered only 0.65–0.85, which led to the conjugate-pair parametrisation.

3.4 The decider, the refinement and the loop

What it does. The loop chooses the structure by raw-data evidence with an Occam term and an outlier-bin mixture, refits the coefficients to the observations by variable projection with terms selected by evidence minus $\frac{1}{2} p \log n$, splits reducible quadratics and conjugate pairs, and grows the degree only under a held-out guard (Section **??** of the main edition).

Explained: how it was tested

Unit tests. `tests/test_loop.py` (5): variable projection reduces the error of close poles, the core flags an unsupported τ-degree, heat does not grow, transport plus diffusion is grouped by factor, a fifth-order law grows once with the held-out guard. `tests/test_general.py` (10): an irreducible cubic in τ is fitted and offered as a candidate; reducible products are split and irreducible heat and transport are not; conjugate closure; the parameter count of a product; models identified across labels. `tests/test_refine_grid.py` (3): grid evidence for real and complex fields, and the same refinement on a grid and on dense points.

Benchmark. Ten named cases $\times$ noise $\{10^{-3}, 10^{-2}, 5 \cdot 10^{-2}\} \times$ sampling {grid, 2000, 400}, 30 runs per cell, against PDE-FIND, weak SINDy, ensemble SINDy, Bayesian SINDy and BOP-DMD. Right factor set: grid 1.00/1.00/0.93, 2000 points 1.00/0.97/0.90, 400 points 1.00/0.93/0.73; at least as good as every baseline in all nine cells. Coefficient cover$_{90}$ 0.75–0.93; weight cover$_{90}$ 0.50–1.00.

Held-out equations (experiment E13). Random equations by family, noise 10^{-2}, 13 draws: right (or equivalent) in 10, one correct abstention, no false-confident answer; on the 10 resolvable draws it is right on 6 against 2 (PDE-FIND) and 4 (weak SINDy). With so few draws these are indications.

What this part does not show

The ten named cases both tuned and measured the defaults. The hand defaults ($P_{\text{conc}} = 0.9$, γ_N floor 0.02, SNR cut 3, outlier cap $3s$) are still hand-set, and the weight intervals are the least calibrated output.

4 The optional learned layers (Phase 4)

In this section

Starting from	the committed pipeline of Section 3, its failure census, and the rule of Section 2.
Why this section	the remaining failures are of three kinds: information absent from the data, information lost by the interpolant, and a search that does not propose the right candidate. Only the last two can be helped by learning, and only as proposals or calibrations.
Role in the paper	these layers were built in a separate session; they exist as code and unit tests and are *not* connected to the loop.

4.1 The simulator: the ground where learned layers are trained and judged

Symbols used here: what they are and where they come from

F1–F5	families: real first order; dispersive; second order in τ; products of 2–3 factors; edge cases	Phase 4 plan, P0.3
draw	one random equation with its true factors and excited modes	`random_pde`
resolvable	enough distinct wavenumbers are present at this noise to identify each factor	scoring rule
correct abstention	ID-KER or UNDECIDED where the information is absent	scoring rule
false-confident	FIT-COMPATIBLE with a wrong set	scoring rule

What it does. `simulator.py` draws a symbol by family from a declared prior with a stability constraint, conjugate pairs for real fields, and a random excitation, so the truth is known by construction. Nothing in the pipeline reads it: it is a held-out test bed. The scoring (`e13_summary.py`) rewards honest abstention: a correct abstention counts as right, a confident wrong answer is the worst outcome.

Explained: how it was tested

`tests/test_simulator.py` (4 tests): every draw is exact (its modes satisfy the dispersion relation), stable and deterministic; the families have their stated shape; a complex factor of a real field is listed with its conjugate; the matching function is scale-aware (right set up to a relative tolerance, not an absolute one).

4.2 L0: learned seeds for the exponential fit

Symbols used here: what they are and where they come from

$s = \lambda T$	a pole made dimensionless by the time span T	`expsum.py`
heatmap	a 67×161 image over $\mathrm{Re}\, s \in [-30, 3]$, $\mathrm{Im}\, s \in [-40, 40]$ with a unit Gaussian at each detectable pole	`expseeds.py`
order head	a classifier for the number of poles, 0 to 8	`expseeds.py`
seed	a proposed set of poles that starts a fit	`SEED_PROVIDER`
CRB	the Cramér–Rao bound: the best possible variance of a rate estimate	core

Why. The matrix pencil is biased at low noise-to-signal ratio and its order choice fails when exponentials merge; the theory has no result for its conditioning ((theory T10), open). This is the one place the theory leaves open for learning.

What it does. A network reads one Fourier bin's time series (real, imaginary and a signal-to-noise channel, 64 samples, zero-padded with a mask) and outputs a heatmap whose peaks are candidate poles and a distribution over the number of poles. It is a four-layer 1-D convolutional encoder followed by two linear layers, with a heatmap head and a count head, trained with a mean-squared heatmap loss plus a cross-entropy on the count. The architecture follows a published design for damped exponentials (Xie, Wakin and Tang, 2021) and its undamped predecessor; the code was reimplemented because the predecessor has no licence and handles undamped lines only. The training data are *generic damped exponential sums* (random order up to 8, near-merged pairs down to spacing 0.3, amplitude ratios down to 10^{-3}, white and coloured noise), *not* PDE solutions: by the Ehrenpreis–Palamodov representation every bin of every in-scope field is such a sum, so the network never sees a PDE family and cannot overfit one. In the pipeline, the proposed pole sets become extra candidates in the per-bin BIC order comparison, with the same refinement budget; `disable()` restores the deterministic pipeline.

Explained: how it was tested

Off and hook tests (`tests/test_learned.py`): the seed hook off is identical to today; the module switch works; a seed can only lower the selected per-bin criterion (junk seeds included); off reproduces the recorded outputs of the unmodified core exactly; a seed never adds an order outside the pencil's range; a seed inside the range is scored (the true pair is selected when the data support two poles, at a tolerance limited by the CRB); with a provider on, every order gets two starts; the null gates (junk seeds on a single pole, pure noise) add no mode; the training draw and its encoding behave.

Experiment E12 (`e12_l0_seeds.py`) is written to compare the network with the pencil plus BIC on held-out sums: order accuracy, rate efficiency against the CRB, false modes, by noise level and pole spacing. *No result file and no trained network were found in the repository*, so there is no evidence yet on whether L0 helps.

Status of L0

The mechanism and its safety (off, junk, null) are tested. Whether the network actually improves order selection or rate error is not measured; the gate of Section 2 has not been run.

4.3 L2: calibration of probabilities and intervals

Symbols used here: what they are and where they come from

$P(\text{top})$	posterior probability of the top structure	loop
isotonic map	a non-decreasing function fitted to turn predicted probabilities into observed frequencies	`ProbabilityCalibrator`
p^*	the Bayes threshold $L_{\text{false}}/(L_{\text{false}} + L_{\text{miss}})$ for a loss pair	`loss_threshold`
inflation	a factor ≥ 1 multiplying an interval width	`IntervalInflator`
Mondrian conformal	a conformal correction computed separately in each regime	`calibrate`

What it does. (i) The probability of the top structure is recalibrated by one isotonic map, which cannot reorder runs and has one degree of freedom per level, so it cannot memorise the case identities. (ii) Decision thresholds are not tuned: once a probability is calibrated, the Bayes decision for a stated loss gives the threshold, which replaces hand values such as P_{conc}. (iii) Interval widths are inflated by one *monotone* map per quantity type on five theory-defined features whose sign the theory fixes (split amplification $\gamma_N^{-1/2}\sigma/|w|$: non-decreasing; mode-count margin: non-increasing; Vandermonde condition: non-decreasing; per-factor signal-to-noise: non-increasing; disagreement between samples: non-decreasing), followed by a Mondrian split-conformal step on a separate calibrate split. Coverage is then guaranteed only within the simulator prior and conditional on correct selection, and every output carries a prior tag.

Explained: how it was tested

`tests/test_learned.py`: the loss threshold; the split into fit, calibrate and test groups is disjoint, nonempty, and raises an error when there are too few groups; isotonic recalibration improves reliability; the interval inflator reaches the nominal coverage and its map is monotone in each feature.

4.4 V0 and V1: the validator

Symbols used here: what they are and where they come from

V0.n	a deterministic diagnostic statistic with a known null	`validate.py`
failure mode	one of ten named ways a run can go wrong (high-κ loss, too few κ, spurious factor, local optimum, noise misspecified, degree too low, out of class, unsplit reducible, weights inseparable, repeated factor)	`FAILURE_MODES`
conformal risk control	choosing one threshold so that the expected fraction of missed true failures is at most α	`_risk_threshold`
OOD gate	a conformal outlier p-value of the Mahalanobis distance of a run's features to the calibration runs	`ood_pvalue`

V0 (deterministic, read-only on a finished run). Implemented: V0.1 posterior-predictive misfit per wavenumber bin (the whitened residual of each bin after projecting on the predicted expo-

nentials is χ^2 under the fitted law); V0.2 the mode-count margin per factor (distinct $|\kappa|$ minus $dx + 1$); V0.3 the evidence gap to the next structure and the structure entropy; V0.10 the split floor (c_F, γ_N, interval width relative to the weight); V0.13 an optional held-out-in-time score. The other eight statistics of the plan are not built.

V1 (learned combiner). One boosted-tree classifier per failure mode (scikit-learn histogram gradient boosting, depth 3, 200 iterations, with monotonic constraints where the sign of a statistic is known) on the V0 vector plus permutation-invariant aggregates of the factor list (number, min/max/mean of the weakest-coefficient z-score, κ-support, inclusion probability, ρ, Vandermonde condition, margin). Identifiers (case name, seed) are never features. It reports a *set* of failure modes chosen by conformal risk control with $\alpha = 0.10$; if the run is out of distribution ($p < 0.01$) the probabilities are withheld and the report says so. It diagnoses only: it never changes a label and never acts.

Explained: how it was tested

`tests/test_validate.py` (2): V0.1 is calibrated when the factors are the truth and flags a run with a missing factor; V0.2 flags too few wavenumbers. `tests/test_learned.py`: the risk control reaches the stated missed-failure rate and the OOD gate flags a shifted run; factor aggregates are invariant to the order of factors; features are built from a loop result; the validator tolerates all-NaN or constant columns. No accuracy of V1 on a real failure-injected corpus was found in the repository.

4.5 The remedy table

What it does. Within one diagnosis the controller would try remedies in an order. The table learns that order offline as the empirical success rate of each remedy on failure-injected simulations (an offline contextual bandit, no online exploration). A success counts only when a wrong output becomes right without adding a false factor on the null set; a remedy that only changes the likelihood is not in the table. The ordering uses the lower credible bound of a Beta$(1, 1)$ posterior, so a remedy seen twice with two successes does not jump ahead of one seen 200 times at 80%. *Tests:* the order, the cost-aware ordering (without a cost a remedy is not favoured).

4.6 What was not built, and why

Layer	Reason
L1a (deterministic residual search)	planned first: the missed high-κ content is raw-data information that a matched-filter scan can recover without learning
L1b (learned residual proposals)	only if an oracle information study shows information present but missed after L1a
Controller	planned: a rule table driven by V0/V1 with the honesty rule that it never changes the likelihood
L6 (amortised posterior)	deferred: its information is already covered by V0/V1 at far less data, and it is valid only in-prior
L5 (learned grouping encoder)	rejected: no stable linear encoder makes the interaction term vanish, and the failures are not grouping failures
Differentiable core	not needed: the refinement on the observations already optimises the raw evidence over the coefficients

Table 2: Layers planned or refused, with the reason recorded in the Phase 4 plan.

5 The final evaluation of the learned layers

In this section	
Starting from	the learned layers of Section 4, the pipeline after its fixes, and corpora of simulated equations with failures injected, all with the truth known.
Why this section	the rule of Phase 4 says a layer ships only if it beats the layer-off pipeline on held-out families. This was tested, layer by layer, in a separate session on a branch of its own.
Role in the paper	the verdict decides what is part of the algorithm: the answer is that the pipeline after its fixes does not need a learned layer.

Symbols used here: what they are and where they come from

AUROC	probability that a detector scores a random failure above a random clean run; 0.5 is chance	detection metric
detection rate	fraction of true failures flagged at a fixed false-alarm rate (here 5%)	conformal gate
oracle ceiling	fraction of failures some remedy rescues if an oracle picked it; an upper bound	controller study
no-go / no-ship	the layer does not pass the gate; its code stays off	rule of Phase 4

Layer	Verdict	Why
V1 failure detector	**no-ship**	leave-one-family-out AUROC 0.85 against 0.62 for V0, and the false alarm stays within its bound, but the detection rate at 5% false alarm is only 0–8% per family (V0 alone: 26–47%); a replay that downgrades flagged runs to UNDECIDED leaves false-confident answers at $7 \to 7$ and loses two correct ones
Exact memo (LB)	**on**	identical label, factors, coefficients and weights in 10/10 paired runs; time ratio 0.995: harmless and almost useless since the core got faster
Learned skip (LB)	no-go	skips almost nothing and harmed 2/165 runs
L2b calibrated $P(\text{top})$	no-go	$P(\text{top}) \geq 0.999$ on 96% of runs: there is no resolution to calibrate
L2c calibrated intervals	no-go	the uncalibrated coverage is already 0.895 at nominal 0.90; both calibrators over-cover with wider intervals
L2d learned constants	no-go	every leave-one-family-out fold picks today's constants
H0 high-κ scan	no-go	never changed an answer in 80/80 and 40/40 paired runs, only added time
Controller C′	not built	oracle ceiling 8/12 failures (67%), mostly by lowering the degree, but V1 as trigger finds 0/12; the other triggers tried (χ^2 tests) find none either
L0 seeds	mechanism only	the safety properties are tested; no trained network and no benefit result exist

Explained: how to read this verdict

Step 1. A layer can help only where the pipeline fails. After the fixes of the main sessions, the failures that remain on the final benchmark are none (30/30) and, on random equations, a handful.

Step 2. The detector V1 is good on average (AUROC 0.85) but a conformal gate needs it to be good in the *tail*: at a false-alarm rate of 5% it catches almost nothing. A detector that cannot flag failures without flagging clean runs cannot be used to downgrade labels.

Step 3. A controller would need a trigger. The oracle shows that a remedy exists for two thirds of the failures (the degree should often be lower), but no diagnosis available at run time points at them: the diagnosis, not the remedy, is the bottleneck.

Step 4. Therefore "pipeline plus ML equals pipeline": the learned layers are an honest negative result, with code, data and gates kept for a future change of the pipeline.

Limits of this evaluation

The corpora are simulated, so the verdicts are within the simulator prior; clean runs include timeouts that carry no features; and the detection numbers use few clean runs per fold. A different pipeline, or a different failure mix, could change them, which is why the code and the gates are kept.

6 Test inventory

In this section	
Starting from	the test files of the repository.
Why this section	to give, in one place, what is covered and by how many tests.
Role in the paper	a reader can see what has a unit test, what has only an experiment, and what has neither.

File	Tests	Covers
`test_core_exact`	6	exact data: factor sets, terms, weights, components sum to data, E_{int} in closed form, the split-ambiguity catalogue
`test_core_v2`	6	exact and noisy factor sets, weights, shared $\kappa = 0$ mode, repeated factor is UNDECIDED, candidate structures
`test_domain`	7	non-periodic cells: extractor, Cadzow denoising, variable-projection gradient, structure on $L = \pi$, two regimes, admissibility, quadtree interface
`test_aggregate_cluster`	8	aggregation, DP, clustering, ARI (Sec. 3.2)
`test_interp`	11	GP (Sec. 3.1)
`test_baselines_bench`	7	baselines, benchmark rows, calibration metrics
`test_phase3`	10	noise, split layer, decider (Sec. 3.3)
`test_loop`	5	loop (Sec. 3.4)
`test_general`	10	general structures: cubic in τ, reducible products, conjugates
`test_refine_grid`	3	refinement on grids
`test_simulator`	4	simulator (Sec. 4.1)
`test_validate`	2	V0 (Sec. 4.4)
`test_learned`	21	L0, L2, V1, remedy (Secs. 4.2–4.4)
`test_ml_contract`	–	entry points, helper signatures, result fields and knobs the learned layers rely on
`test_t77`, `test_t83_half_plane`, `test_t83b_split_finest`, `test_t86`, `test_t94`, `test_t95_smooth_gof`, `test_t97`, `test_t100_path_class_ard`, `test_t101_final_raw_gof`, `test_t103_split_ci_idker`	–	one file per later fix: pooled empty bins, half-plane fit, finest split, raw goodness-of-fit calibration, SVD guard, smooth test, per-branch scales, final goodness of fit, split intervals and identifiability (Sec. **??** of the main edition)
`test_run_loop_pure`, `test_boot_bare_factor`, `test_bench_sigma`, `test_p3_grow`, `test_p4_reinterp`, `test_p8_report`	–	purity of a run, bootstrap of a bare factor, noise conventions of the benchmark, growth and re-interpolation switches, the final report generator

Table 3: Test functions per file; parametrised tests expand to more cases (the repository collects 142 cases in all).

Explained: the full-suite run

Two runs of the whole suite are on record. The first, run for the earlier version of this document on the then working tree: 142 passed, 1 warning, in 19 minutes 34 seconds. The second is the project's own handoff of 2026-11-01, after the later fixes and with more test files (31 files, among them the tests of the fixes T77–T103 and the contract test that keeps the learned layers' entry points stable): 238 passed. I did not rerun the second. The slow part is the non-periodic-cell tests (the quadtree test alone took about 140 s in an older log). An older log of the domain tests had shown one failing quadtree test; it passes in this run. A passing suite shows that the stated properties hold on the cases written for them; it does not by itself show that a layer helps (Section 7).

7 What is established and what is not

Takeaway. **Established** (committed code, tests, benchmark): the classical-learning layers of the pipeline and the loop, with the right factor set recovered in 0.90–1.00 of runs on the named cases down to 2000 points, and 0.73 at 400 points with 5% noise, at least as good as every baseline; GP coverage within the criterion in 11 of 12 cells; the split layer's coverage close to nominal on the tested cases. **Evaluated and not adopted:** V1 (no-ship), the calibrations L2b/L2c/L2d, H0 and the learned skip (no-go); the exact memo is on and changes nothing; L0 has its mechanism tested but no benefit result; the remedy table has no controller to use it. **Not built:** L1a, L1b, the controller, L6. **Open:** generality beyond the named cases (the pipeline's final random-equation test has 20 draws, no false-confident answer, and is still small), and the calibration of the weight intervals.